

Работа с очередями в Laravel (конспект).

Очередь (queue) - это порядок выполнения Задач (job) по принципу: "первым пришёл - первым ушёл".

Настройка подсистемы очередей

В файле `config\queue.php`:

- `beanstalkd` — «легкий» сервер очередей (<https://beanstalkd.github.io/>). Требуется установка:

```
composer require pda/pheanstalk ~4.0
```

- `sqs` — служба Amazon SQS. Требуется установка:

```
composer require aws/aws-sdk-php ~3.0
```

- `sync` — задачи выполняются немедленно. Используется только при отладке;
- `null` — задачи не выполняются. Используется только при отладке;
- `retry_after` - макс. время выполнения задачи (в секундах);
- `block_for` — время между опросами базы данных на предмет появления задачи;
- `failed` — настройки таблицы, хранящей список проваленных задач;
- `dynamodb` — нереляционная база данных Amazon DynamoDB;
- `after_commit` - true - будет ждать, пока не будут зафиксированы транзакции (без транзакций задача будет отправлено немедленно).

```
'redis' => [
    // ...
    'connection' => 'default', // из config/database.connections
    'retry_after' => 90, // макс. время выполнения задачи (в секундах)
],
```

Проверить database драйвер в файле `.env`:

```
QUEUE_CONNECTION=database
```

Создание очереди

```
# создать таблицу в БД
php artisan queue:table

# Если нужно отдельно создать таблицу проваленных задач
php artisan queue:failed-table

php artisan migrate
```

Если задачи планируется хранить в другой базе данных, придется внести правки в код миграций.

Создать задачу (в папке `app/Jobs`):

```
php artisan make:job <имя класса задачи> [--sync]
```

```
# sync - создать неотложное задача
```

Особенности Job

- 1) Реализует интерфейс `Illuminate\Contracts\Queue\ShouldQueue`, помечающий класс как отложенное задача.
- 2) Включает следующие трейты: - `Illuminate\Bus\Queueable` — сериализует объект задачи и записывает в очередь - `Illuminate\Queue\SerializesModels` — сериализует объекты моделей, хранящихся в свойствах объекта задачи (можно удалить) - `Illuminate\Foundation\Bus\Dispatchable` — статические методы, отдающие команды на выполнение задачи; - `Illuminate\Queue\InteractsWithQueue` — позволяет заданию взаимодействовать с очередью (можно удалить).
- 3) Должны присутствовать методы: конструктор и `handle()`.
- 4) Если задачи должны быть уникальными, то нужно реализовать интерфейс `Illuminate\Contracts\Queue\ShouldBeUnique` или `Illuminate\Contracts\Queue\ShouldBeUniqueUntilProcessing` (чтобы задача было разблокировано перед его обработкой)

Сериализуются и сохраняются в очереди только значения общедоступных и защищенных свойств.

Пример:

```
class BbDeleteJob implements ShouldQueue {
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    protected $bb;

    public function __construct(Bb $bb) {
        // $this->bb = $bb;
        $this->bb = $bb->withoutRelations(); // предотвратить сериализацию отношений
    }

    public function handle() {
        if ($this->bb->pic)
            Storage::delete($this->bb->pic);
        $this->bb->delete();
    }
}
// =====

// Запустить эту задачу на выполнение:

use App\Jobs\BbDeleteJob;
...
BbDeleteJob::dispatch($bb);
```

Свойства Job

- `failOnTimeout = true` - задача должно быть помечено как неудачное по таймауту;
- `uniqueFor` - время, пока задача будет уникальным;

- `tries` — количество попыток выполнения задачи;
- `backoff` — время между попытками выполнить задача (в секундах);
- `maxExceptions` — максимальное количество исключений, которое может быть сгенерировано при выполнении задачи;
- `timeout` — максимальное время ожидания завершения выполнения задачи (в секундах). Значение `timeout` должно быть чуть меньше значения `retry_after`;
- `deleteWhenMissingModels` — `true` - не выбрасывать исключение при десериализации объекта модели удалённой записи;

Методы Job

- `uniqueId()` - определить конкретный "ключ", который делает задача уникальным;
- `uniqueVia()` - использовать другой драйвер для получения уникальной блокировки
- `backoff()` — метод, должен возвращать время между попытками выполнить задача (в секундах);
- `retryUntil()` — метод, должен возвращать время следующей попытки выполнить задача, если текущая попытка завершилась неудачей;
- `failed()` - метод для обработки проваленного задачи.

Методы для взаимодействие с очередью (трейт `InteractsWithQueue`)

- `attempts()` — возвращает количество попыток исполнения текущего задачи;
- `release([<задержка>=0])` — вновь помещает текущее задача в очередь (задержка в секундах);
- `fail([$exception])` — помечает задача как проваленное;
- `delete()` — удаляет текущее задача из очереди.

Неотложные задачи (не заносятся в очередь)

Создается с помощью ключа `--sync` :

```
php artisan make:job <имя класса задачи> --sync
```

- не реализует интерфейс `ShouldQueue` ;

- не содержит трейтов `InteractsWithQueue` И `SerializesModels` ;

Запуск отложенных задач-классов (трейт `Dispatchable`)

- `dispatchSync()` — немедленно запускает текущее задача (как неотложное).
- `dispatchNow()` — немедленно запускает текущее задача (как неотложное).
- `dispatch()` — запускает отложенное задача;
- `dispatchIf()` — запуск при условии;
- `dispatchUnless()` — запуск при условии;
- `dispatchAfterResponse()` — запуск после отправки серверного ответа;

Методы результата отправки в очередь:

- `afterCommit()` - позволяет временно включить `after_commit=true`;
- `beforeCommit()` - позволяет временно выключить `after_commit=true`;
- `onQueue(<имя очереди>)` — помещает текущую задачу в очередь с указанным именем;
- `onConnection(<имя службы очередей>)` — помещает текущую задачу в очередь, хранящуюся в службе с указанным именем;
- `delay(<задержка>)` — указывает выполнить задачу по истечении заданной задержки.

```
BbDeleteJob::dispatch($bb)
    ->onQueue('model-processing');
```

Отложенные задачи-функции

В виде функций следует оформлять простые отложенные задачи, выполняемые только в одном месте кода сайта.

```
dispatch(function() {
    Mail::to($user)
        ->send(new AlertMail($user->name));
})->onConnection('database')
    ->onQueue('mail')
    ->afterResponse()
    ->catch(function ($e) {
        // . . .
    });
```

Цепочки отложенных задач

```
BbDeletePrepareJob::withChain([
    new BbFilesDeleteJob($bb),
    function () use ($bb) { . . . },
    new BbRecordDeleteJob($bb)
])->dispatch();
```

Передать конструктору класса ведущего (`BbDeletePrepareJob`) задачи какие-либо параметры невозможно.

С помощью фасада Bus:

```

Bus::chain([
    new ProcessPodcast,
    function () {
        Podcast::update(/* ... */);
    },
])->onConnection('redis')
    ->onQueue('podcasts')
    ->catch(function (Throwable $e) {
        // A job within the chain has failed...
    })
    ->dispatch();

```

Специфические разновидности отложенных задач

Отложенными можно делать слушатели, электронные письма, оповещения, при отправке сохраняя их в очереди, а их отсылку выполнять в отдельном процессе. Для этого они должны

- реализовывать интерфейс `Illuminate\Contracts\Queue\ShouldQueue`;
- содержать трейты `Queueable` и `SerializesModels`.

Свойства и методы: - `queue` — имя очереди, куда будет помещен текущий слушатель; - `connection` — имя службы очередей, где будет храниться текущий слушатель; - `delay` — задержка перед выполнением отложенного слушателя (в секундах); - `shouldQueue()` — если вернет true, текущий слушатель будет выполняться как отложенный - `failed()`

Что бы сделать слушатель-функцию отложенной используйте функцию `queueable()` :

```

use function Illuminate\Events\queueable;
// . . .
Event::listen(queueable(
    function (\Illuminate\Auth\Events\Attempting $event) {
        // . . .
    }))->onQueue('mail')
    ->onConnection('database'));

```

Обычное письмо также можно отправить как отложенное, использовав вместо метода `send()` метод `queue()` или `later(<задержка>, ...)`

```

$mail = new SimpleMail('user')->onQueue('mail')
    ->onConnection('database');
Mail::to('user@bboard.ru')->queue($mail);

```

Отложенные оповещения (используется метод `viaQueues()`):

```
class SimpleNotification extends Notification implements ShouldQueue {
    use Queueable, SerializesModels;
    // . . .
    public function viaQueues() {
        return ['mail' => 'mail-queue', 'slack' => 'slack-queue'];
    }
}
```

События, генерируемые при выполнении отложенных задач

- `Looping` (метод `looping()`) — генерируется перед выборкой отложенного задачи из очереди
- `JobProcessing` (`before()`) — перед выполнением отложенного задачи
- `JobProcessed` (`after()`) — после успешного выполнения отложенного задачи
- `JobExceptionOccurred` (`exceptionOccured()`) — когда в коде отложенного задачи возникает исключение
- `JobFailed` (`failing()`) — когда отложенное задача помечается как проваленное
- `WorkerStopping` (`stopping()`) — перед остановкой процесса-обработчика задач

Пример:

```
class EventServiceProvider extends ServiceProvider {
    // . . .
    public function boot() {
        parent::boot();

        Queue::looping(function () {
            while (DB::transactionLevel() > 0) {
                DB::rollBack();
            }
        })
    }
    // . . .
}
```

Выполнение отложенных задач

```
php artisan queue:listen [<имя службы очередей>]
[--queue=<имя очереди>]
[--timeout=<максимальное время выполнения задачи>]
[--tries=<количество попыток обработки задачи>]
[--delay=<задержка перед обработкой проваленного задачи>]
[--sleep=<время «сна»>]
[--memory=<занимаемый объем памяти>]
[--force]
```

Обработчик, запущенный командой `queue:listen`, отслеживает изменение модулей с программным кодом сайта и в этом случае автоматически перезапускается.

Обработчик можно запустить и другой командой (обеспечивает более высокую производительность):

```
php artisan queue:work [...]
[--once]      # выполнить только одно задача
[--stop-when-empty]  # завершить работу, если очередь «опустела»
```

Обработчик, запущенный командой `queue:work`, не отслеживает правку исходного кода сайта и не перезапускается после этого. Для рестарта используется команда:

```
php artisan queue:restart
```

Работа с проваленными задачами

```
php artisan queue:failed      # вывести список проваленных задач

php artisan queue:retry <обозначения задач>  # попытаться выполнить задачи с указанными обозначениями

php artisan queue:forget <идентификатор задачи>  # удалить задача с заданным идентификатором

php artisan queue:flush      # удалить все проваленные задачи
```



Использование посредников в задачах

Посредник прикрепляется к заданию путём возврата из метода `middleware()`

```
public function middleware()
{
    return [new RateLimited];  # RateLimited - посредник
}
```

Не повторять задача (`dontRelease()`):

```
return [(new RateLimited('backups'))->dontRelease());
```

Если вы используете Redis, вы можете использовать `Illuminate\Queue\Middleware\RateLimitedWithRedis`.

Посредник `Illuminate\Queue\Middleware\WithoutOverlapping` позволяет предотвращать дублирование задач на основе произвольного ключа:

```
return [new WithoutOverlapping($this->user->id)];
```

Указать количество секунд перед повтором (`releaseAfter()`):

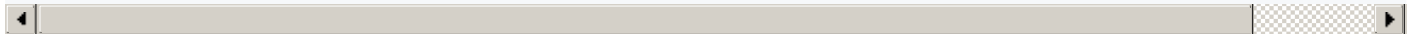
```
return [(new WithoutOverlapping($this->order->id))->releaseAfter(60)];
```

- `expireAfter()` - определить время истечения блокировки с помощью

Регулирование исключений

Посредник `Illuminate\Queue\Middleware\ThrottlesExceptions` позволяет регулировать исключения.

```
# 10 - кол-во исключений, 5 - минуты перед повтором после 10-го исключения
return [(new ThrottlesExceptions(10, 5))->backoff(3)]; # 3 - минуты перед повтором после 1-го иск.
```



```
return [(new ThrottlesExceptions(10, 10))->by('key')]; # 'key' - ключ для кеша
```

Если вы используете Redis, вы можете использовать `Illuminate\Queue\Middleware\ThrottlesExceptionsWithRedis`.

Пакетирование задач

Создание таблицы, содержащей метаинформацию о пакетах задач:

```
php artisan queue:batches-table
```

Чтобы определить пакетизируемое задача нужно добавить трейт `Batchable`. Он предоставляет доступ к методу `batch()`.


```

public function handle()
{
    if ($this->batch()->cancelled()) {
        // ...
        return;
    }
    // ...
}

```

```

$batch = Bus::batch([
    new ImportCsv(1, 100),
    new ImportCsv(101, 200),
    new ImportCsv(201, 300),
])->then(function (Batch $batch) {
    // All jobs completed successfully...
})->catch(function (Batch $batch, Throwable $e) {
    // First batch job failure detected...
})->finally(function (Batch $batch) {
    // The batch has finished executing...
})->name('Import CSV') // именование пакета
->dispatch();

return $batch->id;

```

Добавить задача в пакет:

```

$this->batch()->add(Collection::times(1000, function () {
    return new ImportContacts;
}));

```

Проверка пакетов: - `$batch->id` - имя пакета; - `$batch->totalJobs` - Количество задач, назначенных пакету; - `$batch->pendingJobs` - Количество задач, которые не были обработаны очередью; - `$batch->failedJobs()` - Количество неудачных задач; - `$batch->processedJobs()` - Количество задач, которые были обработаны на данный момент; - `$batch->progress()` - Процент завершения партии (0-100); - `$batch->finished()` - Указывает, завершилось ли выполнение пакета; - `$batch->cancel()` - Отменит выполнение пакета; - `$batch->cancelled()` - Указывает, был ли пакет отменен.

Получить пакет:

```

return Bus::findBatch($batchId);

```

Не отмечать пакет как "отмененный" при неудачной задаче `allowFailures()`:

```

$batch = Bus::batch([
    // ...
])->then(function (Batch $batch) {
    // All jobs completed successfully...
})->allowFailures()->dispatch();

```

Повторная попытка неудачных пакетных задач:

```
php artisan queue:retry-batch <UUID>
```

Запланировать ежедневную обрезку пакетов:

```
$schedule->command('queue:prune-batches --hours=48')->daily();

// -----
// обрезать незавершенные пакетные записи
$schedule->command('queue:prune-batches --hours=48 --unfinished=72')->daily();
```